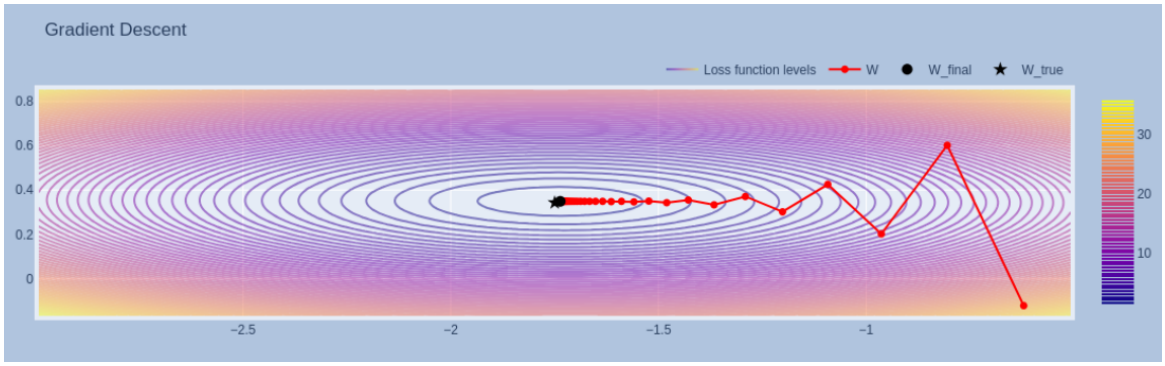
# Лекция 4. Продолжение градиентного спуска. Введение в классификацию

## Изменение вида градиентного шага

Градиентный спуск можно улучшить 2 основными способами. Например, с помощью **momentum**, или «инерции». Идея в том, чтобы учитывать прошлые градиенты и накапливать скорость в направлении движения. Если долго идти в одном направлении, momentum усиливает шаг, помогая пройти через небольшие препятствия (минимумы, которые не являются оптимальными).

Рассмотрим следующее изображение:




На нём мы видим, что по горизонтальной оси — всё хорошо, градиент постепенно шагает в оптимум (на рисунке — в самом центре линий уровня). Однако по вертикальной оси наблюдаются т.н. осцилляции — это когда значение бегает то вверх, то вниз.

Заметим, что если мы просто проссумируем эти векторы, то мы исправим эту проблему и, соответственно, алгоритм будет подбирать значения весов гораздо быстрее.

Формально шаг с momentum вычисляется через вспомогательный вектор $v_t$, который обновляется как комбинация предыдущего шага и текущего градиента:

$$v_t = \gamma v_{t-1} + \eta \nabla J(w_t),$$

а параметры обновляются по формуле

$$w_{t+1} = w_t - v_t.$$

Здесь $\gamma$ — коэффициент «памяти», обычно около 0.9. Чем больше $\gamma$, тем больше

мы хотим учитывать значение предыдущего градиента.

## Градиентный спуск на разреженных данных

Следующая проблема касается только SGD и mini batch GD.

Разреженные данные — данные, в которых определённый признак очень часто принимает нулевое значение. И это далеко не редкая проблема — вспомните метод One Hot Encoding, который представляет категориальные признаки как бинарный вектор.

Нас это беспокоит потому, что при обновлении весов на этих самых нулевых объектах вес не будет изменяться (т.к. частная производная будет равна 0). Значит, скорее всего мы хотим, чтобы на этих объектах был какой-то свой, независимый learing rate, который будет жить своей жизнью.

Давайте посмотрим на одно из решений этой проблемы — AdaGrad:

$$G_{kj} = G_{k-1,j} + (\nabla_w Q(w^{(k-1)}))_j^2$$

$$w_j^{(k)} = w_j^{(k-1)} - \frac{\eta_k}{\sqrt{G_{kj} + \epsilon}}(\nabla_w Q(w^{(k-1)}))_j$$

$(k, j)$ — номер итерации, номер параметра.

Тут уже заметно сложнее. Итак, в матрице $G$ на позиции $kj$ мы храним, насколько много мы уже обучали $w_j$. Накапливаем квадрат градиента, чтобы сильнее подчеркнуть контраст: веса, которые уже сильно изменились, надо изменять поменьше, а те веса, которые изменились мало, надо изменять сильнее.

В зависимости от этого, мы изменяем значение learning rate (тут под шагом подразумевается вся дробь целиком во втором уравнении). Если объект с ненулевым значением признака попадается впервые, то тогда $G_{kj} = 0$ и градиент шагает «со свежей головой».

Тут важно понять, что по факту $G$ есть диагональная матрица (т.е. элементы есть только на диагонали, всё остальное — 0). На первом таком элементе диагонали мы храним, сколько мы шагали для первого объекта выборки, на втором — по второму объекту и т.д.

Проблема: $G_{kj}$ только увеличивается, и замедление шага может оказаться слишком быстрым. Для этого мы воспользуемся модификацией — $\mathrm{RMSProp}$. Отличие лишь в том, что $G_{kj}$ мы теперь вычисляем с некоторым затуханием:

$$G_{kj} = \alpha G_{k-1,j} + (1-\alpha)(\nabla_w Q(w^{(k-1)}))_j^2$$

Кстати, этот метод придумал Джеффри Хинтон — великий человек в области глубокого обучения. Но интереснее, как он его придумал — объясняя на своих занятиях $\mathrm{AdaGrad}$, он просто между строк вкинул, что «можно вот такие вот коэффициенты добавить и будет лучше».

## Введение в классификацию

Задача классификации кажется одной из самых естественных в машинном обучении: у нас есть объект с набором признаков, и нужно определить, к какому классу он относится. Это может быть письмо, которое либо является спамом, либо нет. Это может быть медицинская запись, где нужно понять, болен человек или здоров. Это может быть изображение, на котором может оказаться кошка, собака или что-то ещё.

Формально различают несколько видов классификации. Если классов всего два — говорят о *бинарной классификации*. Если классов больше, задача называется *многоклассовой*.

Целью последующих занятий является формальное определение задачи классификации, решение возникающих трудностей о которых далее), а также вывод алгоритмов, которые способны решать данную задачу.

## Постановка задачи классификации

Итак, в задаче регрессии, постановка была в целом понятной — есть некоторая целевая переменная, которая может принимать любое значение на вещественной прямой. Задача модели — восстановить некоторую функцию, которая бы изящно описывала эти данные, и предсказывать целевую переменную на новых объектах с относительно небольшой погрешностью. То есть:

$$\mathbb{Y} \in \mathbb{R}^d$$

В случае классификации, ситуация сильно упрощается:

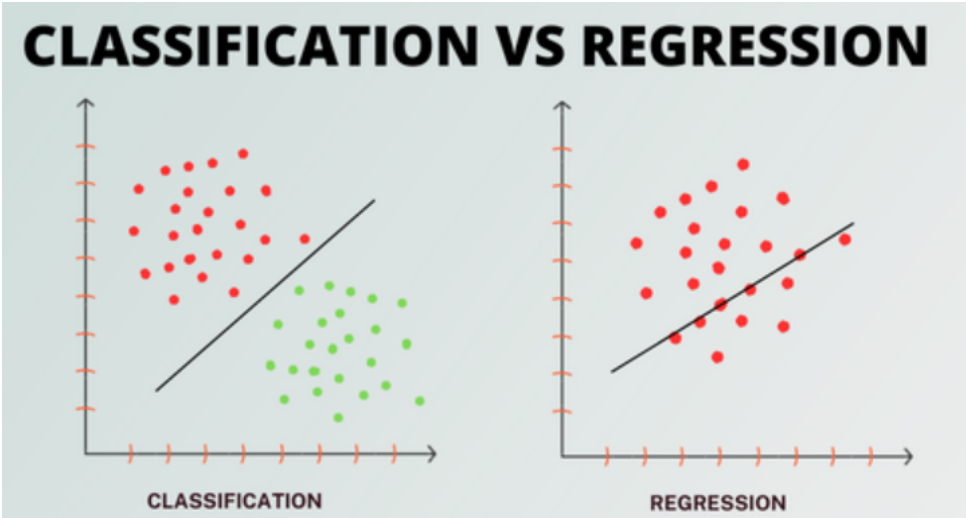
$$\mathbb{Y} \in \{-1, +1\}$$

Будем говорить, что $y = -1$ — *отрицательный ответ*, а $y = +1$ — *положительный ответ*.

То есть, у каждого объекта выборки в качестве целевой переменной выступает некоторый класс — является ли письмо спамом или нет, здоров ли текущий пациент или ему стоит полежать ещё немного и т.д.

Наша текущая задача — построить какую-нибудь простенькую модель, которая способна решать задачу классификации. Но сначала надо определить,

что должна делать функция, которую мы собственно строим. Взгляните на следующее изображение:




Как мы видим, теперь нашей целью является построение такой прямой (гиперплоскости), которая будет в состоянии отделить один класс от другого.

Ну, давайте возьмём уже известную нам модель из задачи регрессии, и просто возьмём её знак:

$$a(x) = sign(\langle w, x \rangle + b)$$

Операция $sign$ возвращает 1 для положительных чисел и -1 для отрицательных. Как раз то, что нам нужно для нашей задачи.

Обучение зададим так:

$$\frac{1}{\ell} \sum_{i=1}^{\ell} [sign(\langle w, x \rangle + b) \neq y_i] -> \min_w$$

Квадратные скобки тут означают *предикат*, т.е. если условие внутри них выполняется - возвращается 1, иначе 0. Соответственно мы считаем количество тех объектов, на которых классификатор сработал некорректно.

Эта функция потерь называется $\text{error rate}$ (доля ошибок). Однако сразу же мы натыкаемся на следующую проблему: невозможность взять производную. Действительно, функция потерь является дискретной, и никакого изменения весов ожидаться не будет. Соответственно, единственный способ решить такую задачу — перебирать вектор весов до тех пор, пока нам не попадётся оптимальный. Задачи, которые решаются только перебором (т.е. нет оптимального

алгоритма решения) называются $\mathrm{NP}$-полные задачи. Сейчас нам нужно придумать, как оптимально решать задачу обучения классификатора.

Чтобы исправить эту ситуацию, введём

$$M = y_i \langle w, x \rangle$$

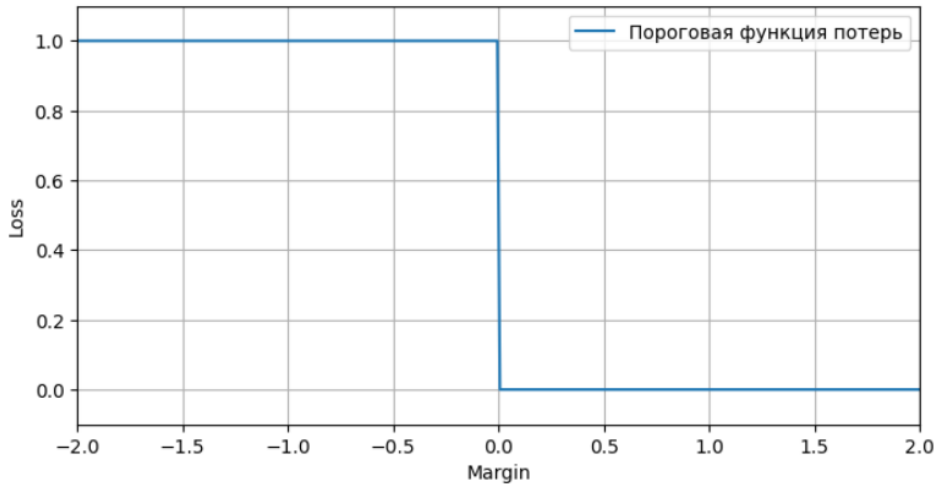
M — $\mathrm{margin}$, или же отступ классификатора.

Можно сказать, что $\mathrm{margin}$ - это уверенность классификатора. Если отступ маленький, т.е. объект находится близко к гиперплоскости, то при минимальном изменении весов класс на этом объекте может поменяться. В таких случаях говорят о неуверенности классификатора. Однако, как правило, если отступ сильно меньше 0, это обычно у выбросов.

Запишем следующую функцию потерь:

$$L(M) = [M < 0]$$

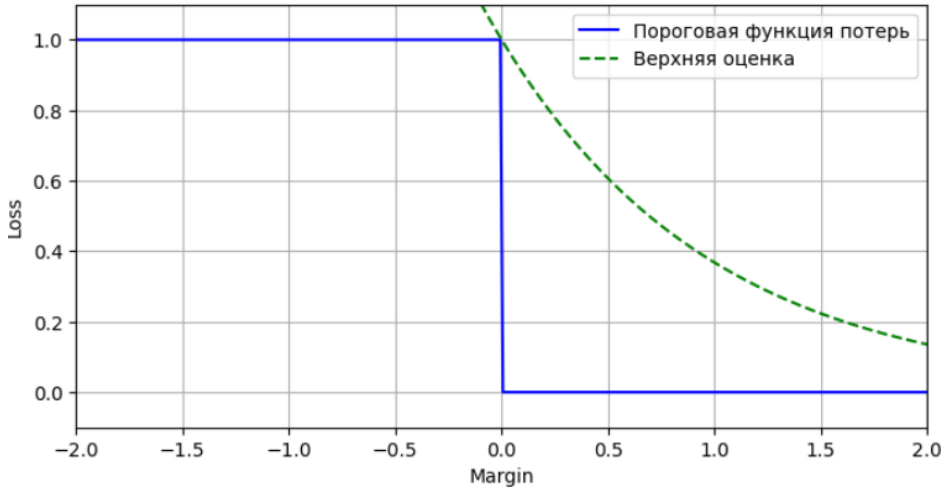
И выведем её график:




Тут всё та же проблема, что и в функции потерь выше: мы никак не сможем посчитать градиент. Однако тут нам открывается интересная возможность — взять какую-то функцию, которая будет похожа на $[M < 0]$, но при этом от неё будет реально взять производную.

Пример такой функции:

$$\tilde{L}(M) = \exp^{-M}$$




Введённая функция называется *верхней оценкой* функции $[M > 0]$. И соответственно, плясать мы будем от этого — вводить новые верхние оценки и обосновывать, почему они действительно имеют смысл.

На дальнейших занятиях мы рассмотрим:

1. $\tilde{L}(M) = \log(1 + \exp^{-M})$ — логистическая функция потерь;
2. $\tilde{L}(M) = \max(0, 1 - M)$ — метод опорных векторов.